



SESHADRIPURAM EDUCATIONAL TRUST

SESHADRIPURAM COLLEGE

POST GRADUATE DEPARTMENT OF COMPUTER APPLICATIONS

#27, Nagappa Street, Seshadripuram, Bengaluru – 560020

PERMANENTLY AFFILIATED TO

**DR. MANMOHAN SINGH BENGALURU CITY UNIVERSITY | AICTE
APPROVED**

ISO 9001:2015 CERTIFIED | NAAC GRADE: A++

department.mca@spmcollege.ac.in | <https://www.spmcollege.ac.in/mca/>

PROJECT REPORT ON

**AutoQA Framework: End-to-End Test Automation using Playwright and
TypeScript**

Submitted in partial fulfillment for the award of degree in

MASTER OF COMPUTER APPLICATIONS (MCA)

IV SEMESTER

Submitted by

PRADEEP KUMAR H N(P18GH24S126076)

Under the Guidance of

Prof. VIJAYALKSHMI S KATTI

Academic Year: 2025-26

1. Introduction

Modern web applications undergo frequent releases, making manual regression testing slow, repetitive, and prone to human error. Test automation addresses this by allowing teams to execute reliable, repeatable test suites quickly and consistently. Playwright, a modern browser automation library developed by Microsoft, combined with TypeScript, provides a fast, type-safe, and cross-browser approach to building robust automated test suites. This project proposes the design and implementation of an end-to-end test automation framework using Playwright and TypeScript to validate the functionality of a web application.

2. Problem Statement

Manual testing does not scale well as an application grows in complexity. Testers spend significant time repeating the same test cases across browsers and releases, increasing the chance of missed defects and delayed feedback to developers. There is a need for an automation framework that is easy to maintain, runs across multiple browsers, integrates with continuous integration pipelines, and provides clear, actionable reports.

3. Objectives

- To design a modular and reusable test automation framework using Playwright and TypeScript.
- To implement the Page Object Model (POM) design pattern for maintainable test scripts.
- To perform cross-browser testing across Chromium, Firefox, and WebKit.
- To integrate API testing alongside UI testing within the same framework.
- To implement data-driven testing using external JSON/CSV data sources.
- To integrate the framework with a CI/CD pipeline for automated test execution on every code change.
- To generate detailed, readable test execution reports.

4. Scope of the Project

The framework will automate core user journeys of a sample web application (such as an e-commerce demo site), including user login, product search, cart operations, form submissions, and checkout flow. It will also validate key backend API endpoints. The scope covers functional and regression testing; performance and security testing are outside the scope of this project.

5. Proposed System / Methodology

The framework will be structured using the Page Object Model, separating locators and page interactions from test logic to improve maintainability. Playwright Test Runner will be used for writing and executing test cases, handling assertions, retries, and parallel execution. Test data will be externalized into JSON/CSV files to support data-driven testing. On failure, Playwright's built-in tracing will capture screenshots and videos to simplify debugging. The suite will be connected to GitHub Actions so that tests run automatically on each commit or pull request, with results published through Allure or Playwright's HTML reporter.

6. System Architecture (High-Level Modules)

- Test Runner Layer: Playwright Test configuration, browser/project setup.
- Page Object Layer: Encapsulates locators and page-specific actions.
- Test Case Layer: Business-flow test scripts using the page objects.
- Test Data Layer: JSON/CSV files for data-driven scenarios.
- Reporting Layer: HTML/Allure reports with screenshots and traces on failure.
- CI/CD Layer: GitHub Actions workflow to trigger and publish test runs.

7. Technology Stack

- Language: TypeScript
- Automation Tool: Playwright
- Runtime: Node.js
- Test Runner: Playwright Test
- CI/CD: GitHub Actions
- Reporting: Allure Report / Playwright HTML Reporter
- Version Control: Git & GitHub

8. Expected Outcome

A working, reusable automation framework capable of executing cross-browser regression tests with minimal manual intervention. The framework will reduce testing time, catch regressions earlier in the development cycle, and produce clear reports that assist in quick defect identification, demonstrating a practical, industry-relevant approach to quality assurance.

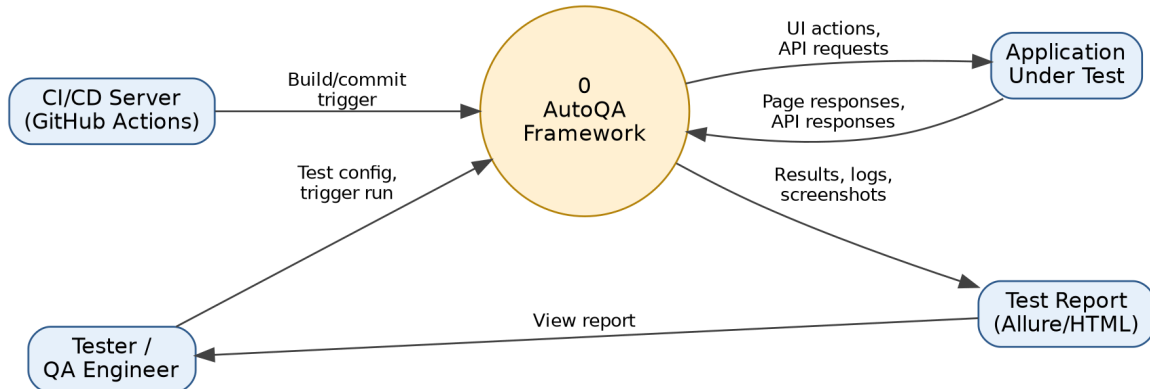
9. Future Enhancements

- Visual regression testing using screenshot comparison.
- Integration with defect-tracking tools such as Jira.
- AI-assisted test case generation and self-healing locators.
- Performance metrics capture during test execution.

10. Data Flow Diagram

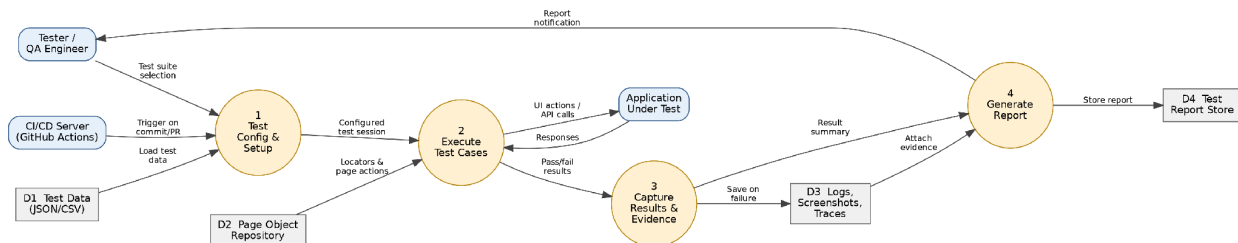
Level 0 DFD (Context Diagram)

The context diagram shows the AutoQA Framework as a single process interacting with external entities: the Tester who configures and triggers runs, the CI/CD server that triggers runs on each commit, the Application Under Test that receives UI/API calls, and the generated Test Report that is viewed by the Tester.



Level 1 DFD (Detailed Process Breakdown)

The Level 1 diagram decomposes the framework into four sub-processes: (1) Test Configuration & Setup, which loads test data and initializes the session; (2) Execute Test Cases, which drives the Application Under Test using page objects; (3) Capture Results & Evidence, which records pass/fail status and saves screenshots/traces on failure; and (4) Generate Report, which compiles results and evidence into a final report for the Tester.



11. Conclusion

This project aims to demonstrate a practical, scalable approach to modern web test automation using Playwright and TypeScript. By combining a maintainable framework design with CI/CD integration, the project will showcase skills directly applicable to real-world QA automation roles.